NAME : Aditya pawar

Branch : AI&DS

Roll NO. : SYB 25

Exp No. 4

AIM : Numerical Computation with NumPy: Perform different computations using NumPy library.

THEORY :

1. Explain the role of NumPy in the Python scientific computing ecosystem.

ANS :

NumPy plays a fundamental role in the Python scientific computing ecosystem by providing support for fast and efficient operations on large multi-dimensional arrays and matrices. It offers a wide collection of mathematical functions that are useful for numerical computation. NumPy also acts as the base library for many other important Python libraries such as Pandas, Matplotlib and Scikit-learn, making it essential for data analysis, scientific research, engineering calculations and machine learning applications.

2. Why is NumPy considered faster than pure Python computations?

ANS :

NumPy is considered faster than pure Python computations because it uses optimized C-language implementations internally and performs operations on entire arrays instead of processing elements one by one. It also requires less memory compared to Python lists and supports vectorized operations, which significantly improves execution speed when working with large datasets.

3. What types of mathematical operations can be performed using NumPy?

ANS :

NumPy supports various mathematical operations such as basic arithmetic operations including addition, subtraction, multiplication and division. It also supports statistical operations like mean, median and standard deviation, matrix operations such as matrix multiplication and transpose, trigonometric functions, logarithmic functions and linear algebra operations. These features make NumPy very useful for numerical and scientific computations.

4. Explain indexing and slicing in NumPy arrays.

ANS :

Indexing in NumPy arrays is used to access individual elements by specifying their position in the array, while slicing is used to extract a subset of elements from an array using a range of indices. NumPy supports indexing and slicing in both one-dimensional and multi-dimensional arrays, which makes it efficient for selecting and manipulating specific portions of large datasets.

5. How does NumPy integrate with libraries like:

ANS :

a. Pandas

NumPy integrates with Pandas by providing the underlying array structure used in Series and DataFrame objects. It helps Pandas perform fast numerical operations and efficient data manipulation during data analysis tasks.

b. Matplotlib

NumPy integrates with Matplotlib by supplying numerical array data that can be used for plotting graphs and visualizations. It helps generate structured data for creating different types of plots and charts.

c. Scikit-learn

NumPy integrates with Scikit-learn by providing array-based input data required for training machine learning models. It supports efficient numerical computation during model building, testing and prediction processes.

```
import numpy as np
# Creating an array
a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])
print("Array A:", a)
print("Array B:", b)
# Arithmetic operations
print("\nArithmetic Operations:")
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
# Statistical operations
print("\nStatistical Operations:")
print("Mean of A:", np.mean(a))
print("Median of A:", np.median(a))
print("Standard Deviation of A:", np.std(a))
```

```
# Aggregate operations
print("\nAggregate Operations:")
print("Sum of A:", np.sum(a))
print("Product of A:", np.prod(a))
# Trigonometric operations
print("\nTrigonometric Operations:")
angles = np.array([0, 30, 45, 60, 90])
print("Sine values:", np.sin(np.radians(angles)))
# Exponential and logarithmic operations
print("\nExponential and Logarithmic Operations:")
print("Exponential:", np.exp(b))
print("Log (natural):", np.log(a))
# Rounding operations
c = np.array([1.2, 2.6, 3.5, 4.9])
print("\nRounding Operations:")
print("Round:", np.round(c))
print("Floor:", np.floor(c))
print("Ceil:", np.ceil(c))
# Maximum and Minimum
print("\nMaximum and Minimum:")
print("Maximum:", np.max(a))
print("Minimum:", np.min(a))
#Indexing and Slicing
print("\nIndexing and Slicing:")
print("First element of A:",a[0])
print("last element of A:", a[-1])
#Sorting
print("\nSorting:")
print("Sorted A:", np.sort(a))
```

```
Array A: [10 20 30 40 50]
Array B: [1 2 3 4 5]

Arithmetic Operations:
Addition: [11 22 33 44 55]
Subtraction: [ 9 18 27 36 45]
Multiplication: [ 10  40  90 160 250]
Division: [10. 10. 10. 10. 10.]

Statistical Operations:
Mean of A: 30.0
Median of A: 30.0
Standard Deviation of A: 14.142135623730951

Aggregate Operations:
Sum of A: 150
Product of A: 12000000

Trigonometric Operations:
Sine values: [0.         0.5        0.70710678 0.8660254  1.        ]

Exponential and Logarithmic Operations:
Exponential: [  2.71828183   7.3890561   20.08553692  54.59815003 148.4131591 ]
```

```
Log (natural): [2.30258509 2.99573227 3.40119738 3.68887945 3.91202301]

Rounding Operations:
Round: [1. 3. 4. 5.]
Floor: [1. 2. 3. 4.]
Ceil: [2. 3. 4. 5.]

Maximum and Minimum:
Maximum: 50
Minimum: 10

Indexing and Slicing:
First element of A: 10
last element of A: 50

Sorting:
Sorted A: [10 20 30 40 50]
```

Conclusion :

NumPy plays an important role in numerical computation by providing efficient support for multi-dimensional arrays and mathematical operations required in scientific computing and data analysis. It improves performance compared to pure Python by using optimized internal implementations and vectorized operations. NumPy allows users to perform various arithmetic, statistical and matrix-related calculations easily, while its indexing and slicing features help in efficient data access and manipulation. Its strong integration with libraries like Pandas, Matplotlib and Scikit-learn makes NumPy a core component of the Python data science ecosystem and an essential tool for performing fast and reliable numerical analysis.